Property Desk

Deployment Guide — Vercel + Supabase

This guide deploys the **Next.js app** (the `frontend/` folder) to **Vercel**, using **Supabase** for the PostgreSQL database and for photo/video storage. The app is self-contained — its own API routes handle extraction, matching, export and login — so it is a complete working app on a single Vercel project.

> **Key fact: one project, and the app works on its own**
>
> You need only **one Vercel project**, with **Root Directory = frontend**. That one project serves both the user interface **and** the backend API.
>
> The separate Python backend (the `backend/` folder / shubham branch) is **not needed** here — it is a duplicate built for the later AWS phase. Vercel ignores it.

What you end up with

```
            Vercel  (one project: Next.js UI + /api/* routes + login)
               |                              |
               v                              v
      Supabase Postgres (data)     Supabase Storage (photos/videos, S3-compatible)
```

Before you start

- A GitHub repo with this code pushed. Deploy the **satyam-frontend** branch; the app lives in its `frontend/` folder.
- A **Supabase** account and a **Vercel** account (free tiers are fine to start).
- Your Anthropic API key for AI extraction (optional — without it the app uses an offline rules extractor that is weaker on Hindi).

Step 1 — Create the Supabase database

- Create a new Supabase project. Pick a region close to your users (e.g. **Mumbai / ap-south-1**). Save the database password.
- Click **Connect** (top bar). You need **two** connection strings:
- **Transaction pooler** (port **6543**) → becomes `DATABASE_URL` (runtime, serverless-safe).
- **Direct connection** (port **5432**) → becomes `DIRECT_URL` (used only for migrations).

Add `?sslmode=require` to **both** URLs. They look like:

```
# DATABASE_URL  (pooled / runtime, port 6543):
  postgresql://postgres.<ref>:<pwd>@aws-0-<region>.pooler.supabase.com:6543/postgres?sslmode=require

# DIRECT_URL    (direct / migrations, port 5432):
  postgresql://postgres:<pwd>@db.<ref>.supabase.co:5432/postgres?sslmode=require
```

Step 2 — Create the Supabase Storage bucket (media)

- Storage → **New bucket** → name it e.g. `property-desk-media`. Keep it **private**.
- Project Settings → **Storage** → **S3 Connection**: copy the endpoint and region, and **generate S3 access keys** (access key id + secret).

```
MEDIA_STORAGE_DRIVER = s3
S3_BUCKET            = property-desk-media
S3_REGION            = <project region, e.g. ap-south-1>
S3_ENDPOINT          = https://<ref>.supabase.co/storage/v1/s3
S3_ACCESS_KEY_ID     = <generated>
S3_SECRET_ACCESS_KEY = <generated>
S3_FORCE_PATH_STYLE  = true
```

Vercel's filesystem is ephemeral, so object storage is required for uploads to persist — do not use the `local` driver in production.

Step 3 — Auth secret and login

Generate a signing secret for the login session cookie:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"   # -> AUTH_SECRET
```

Choose an admin email + password for the dashboard login:

```
AUTH_SECRET          = <the 64-char hex above>
APP_ACCESS_PASSWORD  = <a strong password>   # set => whole app requires login
APP_ADMIN_EMAIL      = you@yourcompany.com
```

Step 4 — Create the Vercel project

Setting	Value
Import	your GitHub repo (Add New -> Project)
Branch	satyam-frontend
Root Directory	frontend (NOT the repo root - the app is in this subfolder)
Framework	Next.js (auto-detected)
Build Command	default (the repo's vercel-build script runs prisma migrate + build)

Add every environment variable from the table on the next page (Production, and Preview if you want preview deploys).

Step 5 — Deploy

Click **Deploy**. On the first build, `prisma migrate deploy` creates all tables in Supabase. When it finishes, open the URL → you are sent to `/login` → sign in with your `APP_ADMIN_EMAIL` / `APP_ACCESS_PASSWORD`.

> **Alternative to build-time migrations**
>
> If you'd rather not migrate on every deploy, set the Vercel Build Command to `next build` and run migrations once from your machine:
>
> `cd frontend && DIRECT_URL="<your direct url>" npx prisma migrate deploy`

Environment variables (set these in Vercel)

Variable	Req	Value
DATABASE_URL	Yes	Supabase pooled URL (6543) + ?sslmode=require
DIRECT_URL	Yes	Supabase direct URL (5432) + ?sslmode=require (migrations)
AUTH_SECRET	Yes	64-char hex from Step 3

Variable	Req	Value
APP_ACCESS_PASSWORD	Yes	dashboard login password
APP_ADMIN_EMAIL	Yes	dashboard login email
AI_PROVIDER	opt	anthropic (or gemini/groq/openrouter; omit for offline rules)
AI_API_KEY	opt	provider key (required if AI_PROVIDER needs one)
AI_MODEL	opt	e.g. claude-sonnet-5-5 (provider default if blank)
MEDIA_STORAGE_DRIVER	Yes	s3 (do not use local on Vercel)
S3_BUCKET / S3_REGION / S3_ENDPOINT	Yes	from Step 2
S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY	Yes	from Step 2 (secret)
S3_FORCE_PATH_STYLE	Yes	true (required for Supabase Storage)
MEDIA_MAX_FILE_MB	opt	default 50; consider 10 on Vercel
APP_URL	opt	your Vercel URL, e.g. https://property-desk.vercel.app
WHATSAPP_*	opt	leave blank until you wire WhatsApp; paste mode works without it

Secrets (DATABASE_URL, DIRECT_URL, AUTH_SECRET, AI_API_KEY, S3_SECRET_ACCESS_KEY) go only in Vercel's env settings — never in the repo. The `.env` file is gitignored.

Known limitations on Vercel (fine for launch)

- **Media served through the function.** Images are fine; very large videos can be slow. Keeping `MEDIA_MAX_FILE_MB` around 10 is safest. Serving media via Supabase signed URLs is a good follow-up.
- **No background worker.** Manual paste runs inline (covered by a 60s function timeout on the AI/export/upload routes). WhatsApp inbound and heavy async are designed for the AWS phase.
- **Cold starts.** The first request after idle is slower. The pooled Supabase URL keeps database connections healthy under serverless.
- **Co-locate regions.** Put the Vercel function region in the same region as Supabase (Project -> Settings -> Functions -> Region) to avoid extra query latency.

Moving to AWS later

The Python backend (`backend/`) is the production target for the next phase: RDS Postgres, S3, an SQS worker, and ECS Fargate. When you get there, point the Next.js frontend at it with `BACKEND_URL=https://<api-domain>` and every `/api/*` request (except `/api/auth/*`) is proxied to it. The runbook is in `backend/README.md` and `docs/BACKEND_API_SPEC.md` (section 2).